

## 05

## Signals &amp; Reactivity

© Gelişim · Junior → Mid-Level

## 🔗 Bu modülde ne öğreneceğiz?

- ✓ Signal'in ne olduğunu ve reaktifliğin motorunun neden signal olduğunu
- ✓ `signal`, `computed`, `effect` ile durumu yönetmeyi (çekirdek API)
- ✓ `linkedSignal` ile “yerel çalışma kopyası” kurmayı
- ✓ `resource` ile asenkron veriyi signal dünyasına taşımayı
- ✓ Signal sorgularıyla (`viewChild`, `viewChildren`) şablondaki elemanlara erişmeyi

5.1 Signal Nedir?

Proje

Özet

5.2 Signal API

5.3 linkedSignal

5.4 resource

5.5 Signal Queries

Mini

## 💡 Signal Nedir?

Önceki modüllerde signal'leri zaten kullandık; şimdi **ne olduklarını** tam oturtalım. Çünkü signal, Angular'da arayüzün canlı kalmasını sağlayan motordur.

### Nedir?

**Signal, bir değeri tutan ve o değer değiştiğinde kimlerin etkilendiğini bilen “akıllı bir kutudur”.** Sıradan bir değişkenden farkı şudur: sıradan değişken değişince kimse haberdar olmaz; signal değişince, onu kullanan her yer (ekran, hesaplamalar) otomatik güncellenir.

#### 💡 Excel benzetmesi

Bir signal, Excel'deki bir **hücre** gibidir. A1 hücresine bağlı bir formül (“=A1\*2”) yazdıysan, A1 değişince o formül kendiliğinden yeniden hesaplanır. Signal'ler de böyle: değer değişir, ona bağlı her şey kendiliğinden tazelenir.

### Neden var? (Hangi derde deva?)

Bir uygulamada en zor işlerden biri şudur: **veri değişince ekranı doğru ve verimli biçimde güncel tutmak.** Signal bunu çözer; Angular hangi signal'in nerede kullanıldığını tam olarak bildiği için, bir değer değiştiğinde **sadece o değere bağlı yerleri** günceller — gereksiz iş yapmaz. Hızlı ve öngörülebilir.

### Nasıl? (kısaca — detay sonraki derste)

TYPESCRIPT

```
import { signal } from '@angular/core';

const count = signal(0); // signal oluştur, başlangıç değeri 0

count(); // OKUMA -> 0 (fonksiyon gibi parantezle)
count.set(5); // YAZMA -> artık 5
```

Okuması neden parantezli? Çünkü signal bir **fonksiyondur**; onu çağırdığında hem güncel değeri verir hem de “seni dinliyorum” kaydını tutar. Bütün API'yi bir sonraki derste derinlemesine göreceğiz.

#### ✓ Bu Dersten Çıkarılacaklar

- ✓ Signal = değerini tutan ve değişince bağlı her yeri otomatik güncelleyen akıllı kutu.
- ✓ Sıradan değişkenden farkı: değişimi “duyurur”; Angular yalnızca etkilenen yerleri tazeler.
- ✓ Okuma `count()` (parantezli), yazma `count.set(...)`.
- ✓ Reaktifliğin (canlı arayüzün) temel yapı taşıdır.

## <> Signal API: signal, computed, effect

Bu, modülün **kalbi**. Üç temel araç var ve hepsini bol örnekle göreceğiz: durumu tutan **signal**, türetilmiş değer üreten **computed**, ve yan etki çalıştıran **effect**.

### signal() — Durumu Tutmak

Bir signal oluşturur, okur ve değiştirirsin. İki yazma yöntemi vardır:

TYPESCRIPT

```
import { signal } from '@angular/core';

const count = signal(0);

// Okuma
console.log(count());           // 0

// set: doğrudan yeni değer ver
count.set(10);                  // 10

// update: mevcut değere göre hesapla
count.update((n) => n + 1);     // 11
```

**set** ile yeni değeri doğrudan verirsin; **update** ile **mevcut değere bakarak** yeni değeri üretirsin (sayaç artırma, listeye ekleme gibi).

### Nesne ve Dizilerde: Yeni Referans Ver (immutability)

Bir signal nesne ya da dizi tutuyorsa, onu **içeriden değiştirme** (mutasyon). Bunun yerine **yeni** bir nesne/dizi ver. Yoksa signal “değer değişti” demez ve ekran güncellenmez.

TYPESCRIPT

```
const tasks = signal<string[]>([]);

// YANLIŞ: aynı diziyi içeriden değiştirmek
tasks().push('Süt al');           // signal değişimi fark etmez!

// DOĞRU: yeni dizi ver (spread ile)
tasks.update((list) => [...list, 'Süt al']);
```

### ⚠ Neden mutasyon çalışmaz?

Signal, “değer değişti mi?” sorusuna referansa bakarak cevap verir. `push` aynı diziye (aynı referansı) kullanır; signal hiçbir değişiklik görmez. Yeni bir dizi ( `[...list, x]` ) verince referans değişir, signal tetiklenir. Bu yüzden Todo uygulamasında `map` / `filter` / `spread` kullandın.

## computed() — Türetilmiş Değer

**Nedir?** Başka signal'lerden **hesaplanan**, salt-okunur bir signal. **Neden?** Aynı hesabı elle birçok yerde tekrarlamamak ve kaynak değişince otomatik güncel kalmak için.

TYPESCRIPT

```
import { signal, computed } from '@angular/core';

const price = signal(100);
const quantity = signal(3);

// price veya quantity değişince otomatik yeniden hesaplanır
const total = computed(() => price() * quantity());

total(); // 300
quantity.set(5);
total(); // 500 (kendiliğinden güncellendi)
```

İki önemli özelliği var: **tembel** (lazy) — kimse okumuyorsa hesaplanmaz; ve **önbellekli** (memoized) — kaynaklar değişmedikçe aynı sonucu yeniden hesaplamaz. Yani hem rahat hem hızlı. Bir `computed` 'a **elle değer atayamazsın**; o, kaynaklarına bağlıdır.

### 🔒 Kural

Bir değer **başka signal'lerden hesaplanıyorsa**, onu ayrı bir signal'de tutup elle güncelleme — `computed` kullan. Daha az hata, her zaman tutarlı.

## effect() — Yan Etki Çalıştırmak

**Nedir?** Bağımlı olduğu signal'ler değiştiğinde otomatik çalışan bir fonksiyon. **Neden?** Signal değişimine tepki olarak **reaktif olmayan bir iş** yapman gerektiğinde: bir şeyi loglamak, `localStorage` 'a yazmak, bir grafik kütüphanesini güncellemek gibi.

TYPESCRIPT

```
import { signal, effect } from '@angular/core';

const theme = signal<'light' | 'dark'>('light');
```

```
effect(() => {  
  // theme her değiştiğinde otomatik çalışır  
  document.body.dataset['theme'] = theme();  
});  
  
theme.set('dark'); // effect tetiklenir, body güncellenir
```

#### ⚠ effect ile computed 'i karıştırma

Bir **değer üretmek** istiyorsan **computed** kullan. **effect** yalnızca **dış dünyaya bir şey yapmak** içindir (log, depolama, DOM API'si). Yeni başlayanların sık hatası: **effect** içinde başka bir signal'i **set** ederek değer üretmeye çalışmak — bunun yerine neredeyse her zaman **computed** doğru araçtır.

## untracked() — Bağımlılık Yaratmadan Okumak

Bazen bir **computed** / **effect** içinde bir signal'i **okumak ama ona bağımlı olmamak** istersin (o değişince yeniden çalışmasın). **untracked** bunu sağlar:

TYPESCRIPT

```
effect(() => {  
  const current = value(); // buna bağımlı (değişince çalışır)  
  const id = untracked(() => userId()); // sadece oku; userId değişimi tetiklemez  
  log(id, current);  
});
```

### 🔗 Mini Quiz

1. **set** ile **update** farkı nedir?  
↳ set doğrudan yeni değeri verir; update mevcut değere bakarak yeni değeri üretir.
2. Bir signal dizi tutuyorsa neden **push** yerine yeni dizi verirsin?  
↳ Signal değişimi referansa bakarak anlar; push aynı referansı kullanır, fark edilmez. Yeni dizi referansı değiştirir.
3. **computed** ne zaman kullanılır ve neden ona değer atanamaz?  
↳ Bir değer başka signal'lerden hesaplanıyorsa. Salt-okunurdur çünkü değeri tamamen kaynaklarına bağlıdır.
4. **effect** ne için vardır, **computed** 'dan farkı ne?  
↳ Signal değişince reaktif-olmayan bir yan etki (log, depolama, DOM) çalıştırmak için. computed değer üretir; effect dışarıya iş yapar.

### ✗ Sık Yapılan Hatalar

- ✗ Nesne/diziyi içeriden değiştirmek ( **push** , alan atama). Yeni referans ver; yoksa güncellenmez.

- ✗ **Türetilmiş değeri ayrı signal'de elle güncellemek.** `computed` kullan; tutarlılığı garanti eder.
- ✗ **`effect` içinde değer üretmek/signal `set` etmek.** Genelde yanlış; `computed` 'a geç.
- ✗ **Signal'i parantezsiz okumak.** `count` değil `count()`.

### ✓ Bu Dersten Çıkarılacaklar

- ✓ `signal()` : durumu tutar; `set` (doğrudan) / `update` (mevcuda göre).
- ✓ Nesne/dizilerde immutability: hep yeni referans ( `[...]` , `map` , `filter` ).
- ✓ `computed()` : türetilmiş, salt-okunur, tembel ve önbellekli; kaynak değişince otomatik güncel.
- ✓ `effect()` : yan etki (log/depolama/DOM); değer üretmek için değil.
- ✓ `untracked()` : bir signal'i bağımlılık yaratmadan okumak için.

## Ders 5.3

# linkedSignal

`computed` 'i öğrendin: türetilmiş ama **salt-okunur**. Peki ya başka bir signal'den türetilen **ama yine de elle değiştirebileceğin** bir değere ihtiyacın olursa? İşte `linkedSignal` tam buradadır.

## Nedir?

`linkedSignal` , bir kaynaktan türetilen AMA üzerine yazılabilen bir signal'dir. `computed` gibi bir kaynağa bağlıdır; ama `computed` 'in aksine `set` / `update` ile değiştirebilirsin. Kaynak değiştiğinde ise yeniden hesaplanıp **sıfırlanır** (senin yaptığın yerel değişiklikler kaynağın yeni değeriyle değişir).

## Neden var? (Hangi derde deva?)

Klasik senaryo: bir **yerel çalışma kopyası**. Diyelim bir listeden bir öge seçiliyor ve sen onu düzenliyorsun. Seçili öge değişince düzenleme alanının **otomatik sıfırlanmasını** istersin. `linkedSignal` bunu kendiliğinden yapar.

## Nasıl?

TYPESCRIPT

```
import { signal, linkedSignal } from '@angular/core';

const products = signal(['Kahve', 'Çay', 'Su']);

// Kaynak (products) değişince ilk ögeye geri döner;
// ama aradnında elle de değiştirebilirsin.
const selected = linkedSignal(() => products()[0]);

selected(); // 'Kahve'
selected.set('Çay'); // elle değiştir -> 'Çay'
```

```
products.set(['Limonata', 'Ayran']);
selected(); // 'Limonata' (kaynak deðiřti -> sıfırlandı)
```

## computed mu, linkedSignal mı?

| Soru                                                  | Araç         |
|-------------------------------------------------------|--------------|
| Değer tamamen kaynaktan türeniyor, elle değıřmeyecek  | computed     |
| Kaynaktan türeniyor AMA elle de değıřtirebilmem lazım | linkedSignal |
| Tamamen bağımsız, kendi başına durum                  | signal       |

### Q Kısa kural

“Türetilmiş + salt-okunur” ise **computed** ; “türetilmiş + elle de yazılabilir + kaynak değıřince sıfırlansın” ise **linkedSignal** .

### Q Mini Quiz

- linkedSignal** ’in **computed** ’den temel farkı nedir?  
↳ linkedSignal türetilmiş olmasına rağmen elle değıřtirilebilir (yazılabilir).
- Kaynak değıřtiğinde **linkedSignal** ’in değıřine ne olur?  
↳ Yeniden hesaplanır ve sıfırlanır; senin elle yaptığın değıřiklik kaynağın yeni değıřiyle değıřir.
- “Yerel çalışma kopyası” için hangisini seçersin?  
↳ **linkedSignal** .

### × Sık Yapılan Hatalar

- × Sırf “yazabileyim” diye **computed** yerine ayrı **signal** + **effect** kurmak. **linkedSignal** bu deseni tek satırda verir.
- × Kaynaktan kopuk bir **signal** kullanıp elle senkron tutmaya çalışmak. Kaynak değıřince sıfırlama gerekiyorsa **linkedSignal** kullan.

### ✓ Bu Dersten Çıkarılacaklar

- ✓ **linkedSignal** = kaynaktan türeyen ama yazılabilen signal.
- ✓ Kaynak değıřince yeniden hesaplanıp sıfırlanır (yerel düzenlemeler gider).
- ✓ “Yerel çalışma kopyası” senaryoları için idealdir (seçili öğeyi düzenleme vb.).
- ✓ Salt-okunur türetme **computed** ; yazılabilir türetme **linkedSignal** .

## resource: Asenkron Veri

Şu ana kadarki signal araçları **senkrondu**: değer hemen oradaydı. Ama gerçek uygulamalar veriyi çoğu zaman **sunucudan** alır ve bu **zaman alır** (asenكرون). **resource**, bu asenkron veriyi signal dünyasına taşıyan araçtır.

### Nedir?

**resource**, asenkron bir işi (örneğin sunucudan veri çekmeyi) sarmalayıp sonucunu ve durumunu signal olarak veren bir yapıdır. Sana hazır biçimde şunları sunar: veri ( **value** ), yükleniyor mu ( **isLoading** ), hata var mı ( **error** ).

### Neden var? (Hangi derde deva?)

Asenkron veriyi elle yönetmek zahmetlidir: “yükleniyor” bayrağı tut, hatayı yakala, parametre değişince yeniden iste, eski isteği iptal et... **resource** bunların hepsini üstlenir. Sen sadece **neye bağlı olduğunu** ve **nasıl getirileceğini** tarif edersin.

### Nasıl?

TYPESCRIPT

```
import { signal, resource } from '@angular/core';

const userId = signal(1);

const userResource = resource({
  // params: REAKTİF kısım. İçindeki signal değişince loader yeniden çalışır.
  params: () => ({ id: userId() }),

  // loader: asıl asenkron iş. params'ı alır, veriyi döndürür.
  loader: async ({ params }) => {
    const res = await fetch(`/api/users/${params.id}`);
    return res.json();
  },
});
```

Sonucu ve durumunu signal olarak okursun — şablonda doğrudan kullanılabilir:

TYPESCRIPT

```
userResource.value(); // gelen veri (yoksa undefined)
userResource.isLoading(); // true/false
userResource.error(); // hata (varsa)
```



```
userResource.status();    // 'idle' | 'loading' | 'resolved' | 'error' ...
userResource.reload();    // elle yeniden getir
```

### ⚠ En kritik kural: `params` reaktiftir, `loader` değil

Yeniden getirilmesini istediğin signal'leri `params` içinde oku. `loader` içinde okuduğun signal'ler **takip edilmez** (değişince yeniden istek gelmez). Kural: bağımlılıklar `params` 'ta, iş `loader` 'da.

## Loader'ın Diğer İki Girdisi: `abortSignal` & `previous`

Loader aslında üç şey alır: `{ params, previous, abortSignal }`. `abortSignal`, tarayıcının iptal sinyalıdır; `fetch` 'e verirsin. Böylece `params` değişip yeni istek başlayınca, hâlâ süren eski istek **iptal edilir** — hızlı değişimlerde eski cevabın yenisinin üstüne düşmesini (race) önler. `previous` ise önceki durumu tutar (yeni veri gelene kadar eskisini göstermek gibi).

TYPESCRIPT

```
loader: async ({ params, abortSignal }) => {
  const res = await fetch(`/api/users/${params.id}`, { signal: abortSignal });
  return res.json();
},
```

## Durumlar ve Güvenli Okuma

`status()` şu değerleri alır: `idle` (istek yok), `loading`, `reloading` (yeniden yükleme), `resolved` (geldi), `error`, `local` (elle set edildi). Bir de çok önemli bir güvenlik kuralı var:

### ⚠ Hata durumunda `value()` okumak istisna FIRLATIR

Resource hata durumundayken `value()` okumak çalışma anında hata atar. Bu yüzden önce `hasValue()` ile koru (hem `undefined` 'ı eler hem hatayı yakalar) ya da `error()` 'ı kontrol et.

Şablonda temiz, “undefined jimnastiği” yapmayan kalıp şudur:

HTML

```
@if (userResource.isLoading()) {
  <app-skeleton />
} @else if (userResource.error()) {
  <p>Hata oluştu. <button (click)="userResource.reload()">Tekrar dene</button></p>
} @else if (userResource.hasValue()) {
  <app-user-card [user]="userResource.value()" />
}
```

## Kısayol: `httpResource`

Veriyi HTTP ile çekiyorsan daha da kıyası var: `httpResource`. Ona bir URL döndüren fonksiyon verirsin; içindeki signal değişince isteği otomatik yeniler:

```
import { httpResource } from '@angular/common/http';

const userId = signal(1);

// userId değişince istek otomatik yenilenir
const user = httpResource(() => `/api/users/${userId()}`);

user.value(); // gelen kullanıcı
user.isLoading(); // yükleniyor mu
```

### 📖 Detayı ileride

HTTP iletişiminin (provider kurulumu, başlıklar, hata yönetimi) tamamını **Modül 11**'de göreceğiz. Burada amaç, asenkron verinin signal olarak nasıl ifade edildiğini kavramak.

## Hangisini Ne Zaman?

| Durum                                           | Tercih                        |
|-------------------------------------------------|-------------------------------|
| Basit HTTP GET istekleri                        | <code>httpResource</code>     |
| Özel <code>fetch</code> / HTTP dışı asenkron iş | <code>resource</code>         |
| Veri değiştirme (POST/PUT/DELETE)               | Hiçbiri — doğrudan HttpClient |

Son satır önemli: `resource/httpResource` **okuma** içindir; veri değiştiren işlemler için doğrudan HttpClient kullanılır (Modül 11).

### 🔗 Mini Quiz

- `resource` sana hangi hazır signal'leri verir?  
↳ `value()` (veri), `isLoading()`, `error()`, `status()` (+ `reload()`).
- Bir signal değişince yeniden veri çekilmesini istiyorsan onu nereye koymalısın?  
↳ `params` fonksiyonunun içine (reaktif kısım).
- `httpResource` ne işe yarar?  
↳ HTTP için kısayol: bir URL döndüren fonksiyon alır, signal değişince isteği otomatik yeniler.

### × Sık Yapılan Hatalar

- × **Bağımlılığı** `loader` içinde okumak. Takip edilmez; `params` 'a koy.
- × “Yükleniyor” ve “hata” durumlarını elle signal'lerle yönetmek. `resource` bunları zaten verir.
- × `value()` 'yu kontrolsüz kullanmak. Veri gelmeden `undefined` olabilir; `isLoading` / `error` ile birlikte düşün.

### ✓ Bu Dersten Çıkarılacaklar

- ✓ `resource` : asenkron veriyi signal olarak sunar (value/isLoading/error/status + reload).
- ✓ `params` reaktiftir (bağımlılıklar buraya); `loader` asıl işi yapar.
- ✓ `httpResource(() => url)` : HTTP için kısayol; URL'deki signal değişince otomatik yeniler.
- ✓ Yükleniyor/hata/iptal gibi zahmetli işleri senin yerine üstlenir.

## Ders 5.5

# 🎯 Signal Queries: viewChild & viewChildren

Bazen şablondaki bir HTML elemanına ya da bir alt component'e **doğrudan TypeScript'ten** erişmen gerekir: bir input'a odaklanmak, bir alt component'in metodunu çağırmak gibi. Bunu **signal sorgularıyla** yaparsın.

## Nedir & Neden?

`viewChild` , şablonundaki **tek bir** elemana/component'e erişip onu **signal olarak** verir. `viewChildren` ise **birden çok** eşleşeni signal olarak verir. Neden signal? Çünkü diğer her şeyle (computed/effect) uyumlu biçimde kullanabilirsin.

## Nasıl?

Şablonda hedefe bir referans ( `#name` ) verirsin, sınıfta `viewChild` ile alırsın:

TYPESCRIPT

```
import { Component, viewChild, ElementRef, effect } from '@angular/core';

@Component({
  selector: 'app-search',
  template: `<input #box class="search" placeholder="Ara..." />`,
})
export class Search {
  // #box referansına erişim (signal olarak)
  box = viewChild<ElementRef<HTMLInputElement>>('box');
```

```
constructor() {  
  effect(() => {  
    // eleman hazır olduğunda odaklan  
    this.box()?.nativeElement.focus();  
  });  
}
```

Zorunlu olmasını istiyorsan `viewChild.required(...)` dersin (o zaman değer hiç `undefined` olmaz).  
Birden çok eleman için `viewChildren` bir **dizi** döndürür:

TYPESCRIPT

```
items = viewChildren<ElementRef>('item');  
// items() -> ElementRef[] (kaç tane #item varsa)
```

### 🔗 Önce şablon, sonra TS

Çoğu işi binding'lerle ( `[value]` , `(click)` , `@if` ) yaparsın; doğrudan elemana erişim **istisnadır** (odaklanma, ölçüm, bir kütüphane elemana ihtiyaç duyuyorsa). “Bunu binding ile çözebilir miyim?” diye sor; çözemiyorsan `viewChild` kullan.

### 🔗 Mini Quiz

1. `viewChild` ne döndürür ve neden signal'dir?  
↳ Şablondaki tek bir elemana/component'e erişimi signal olarak döndürür; böylece computed/effect ile uyumludur.
2. `viewChild` ile `viewChildren` farkı nedir?  
↳ `viewChild` tek eşleşen verir; `viewChildren` tüm eşleşenleri dizi olarak verir.
3. Değerin hiç `undefined` olmamasını nasıl garanti edersin?  
↳ `viewChild.required(...)` kullanarak.

### ✗ Sık Yapılan Hatalar

- ✗ **Binding ile çözülebilecek şeyi `viewChild` ile yapmak.** Önce binding'leri düşün; doğrudan erişim istisnadır.
- ✗ **Değer gelmeden kullanmak.** Eleman hazır olmayabilir; `?.` ile koru veya `required` kullan.

### ✓ Bu Dersten Çıkarılacaklar

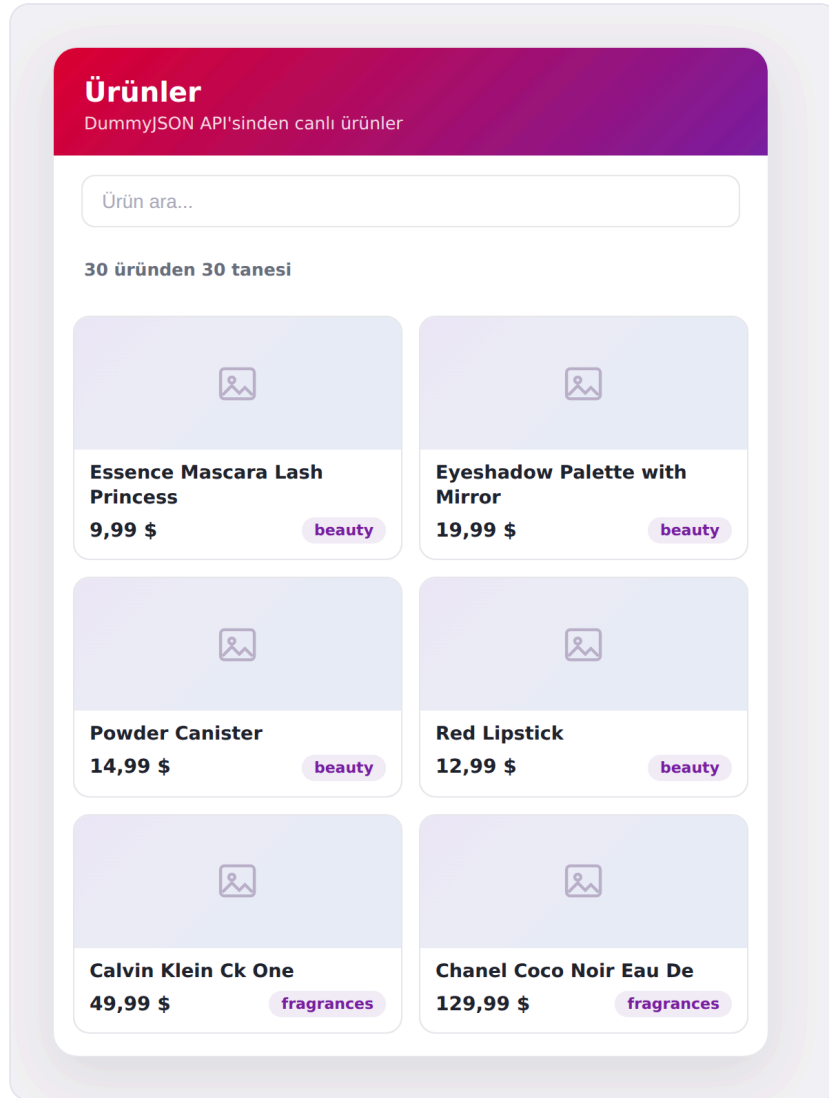
- ✓ `viewChild` : şablondaki tek elemana/component'e signal erişimi.

- ✓ `viewChildren` : tüm eşleşenleri dizi signal olarak verir.
- ✓ `#name` referansı + sınıfta `viewChild('name')`.
- ✓ Doğrudan erişim istisnadır; çoğu işi binding'lerle yap.

## Bölüm

# 🚩 Mini Proje: Ürünler — Liste & Detay (DummyJSON)

Bu modülün kavramlarını GERÇEK bir API ile birleştir: açık ve ücretsiz **DummyJSON** API'sinden ürünleri getiren bir **liste** sayfası ve bir ürüne tıklayınca id'ye göre açılan **detay** sayfası. Böylece `httpResource` 'u iki farklı yerde kullanırsın: listede ve id'ye göre detayda.

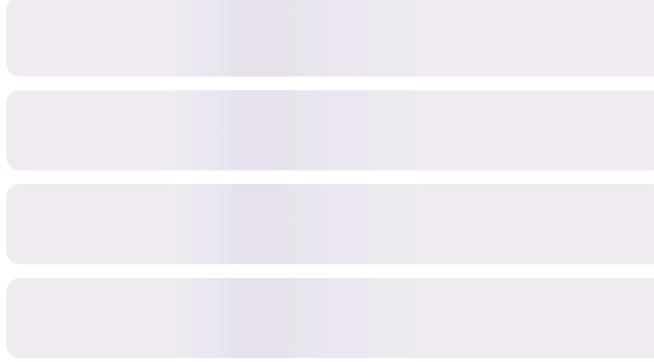


Liste — sonuç durumu (httpResource ile gelen ürünler)

## Ürünler

DummyJSON API'sinden canlı ürünler

Yükleniyor...



Liste — yükleniyor durumu (isLoading)

← Ürünlere dön



ESSENCE

**Essence Mascara Lash Princess**

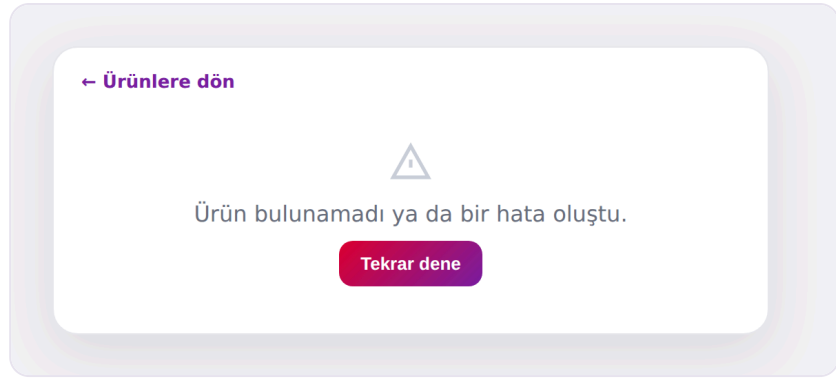
**9,04 \$** 9,99-\$ %9,5 indirim

★★★★☆ 2,56 / 5

beauty Stokta (99 adet)

The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.

Detay — tam görünüm: görseller, fiyat/indirim, puan, stok, açıklama



Detay — hata durumu (error + Tekrar dene)

## API uç noktaları (DummyJSON)

| Amaç                  | Uç nokta                                                                              |
|-----------------------|---------------------------------------------------------------------------------------|
| Ürün listesi          | <a href="https://dummyjson.com/products">https://dummyjson.com/products</a>           |
| Tek ürün (id'ye göre) | <a href="https://dummyjson.com/products/{id}">https://dummyjson.com/products/{id}</a> |

### 📄 Auth yok, CORS açık

DummyJSON ücretsizdir, anahtar/oturum gerektirmez ve tarayıcıdan doğrudan çağrılabilir. Yalnızca okuma yapıyoruz; yazma işlemleri simüledir (kalıcı olmaz), ama bize gerekmiyor.

## Özellikler / Gereksinimler

- **Liste sayfası:** ürünler `httpResource` ile gelsin; **yükleniyor** ve **hata** durumları gösterilsin.
- **Arama:** gelen listeyi `computed` ile **yerel** süz (her harfte sunucuya gitme).
- **Detaya geçiş:** bir ürüne tıklayınca `/product/:id` rotasına git.
- **Detay sayfası:** id'ye göre `httpResource` ile ürünü getir; görselleri, fiyatı/indirimi, puanı, stok ve açıklamayı göster; yükleniyor/hata olsun.
- **Geri dönüş:** detaydan listeye dönülebilsin.

## Angular tarafı (sende) — ipuçları

- **Liste:** `products = httpResource(() => 'https://dummyjson.com/products')` (DummyJSON `{ products: [...] }` döndürür; tipini ona göre kur).
- **Arama:** `searchTerm = signal('')` + `filtered = computed(() => ...)` ile gelen listeyi süz.
- **Detay id:** rotadan id'yi bir signal olarak al; `product = httpResource(() => `.../products/${id()}`)` — id değişince otomatik yeniden çeker.
- **Durumlar:** iki sayfada da `@if (x.isLoading()) ... @else if (x.error()) ... @else if (x.hasValue()) ...` kalıbı.
- **Kurulum:** `httpResource` için `app.config.ts` 'te `provideHttpClient()` ekle.
- **Routing (minimal):** iki rota — liste ve `product/:id`; kartta `routerLink`, detayda id'yi rotadan oku. Tam anlatım Modül 9'da.

## ❓ Materyaller

Statik referansı ayrıca verdim (JS yok): **product-list.html**, **product-detail.html**, **styles.scss**, **styles.css** ve ekran görüntüleri. Görseller yer tutucudur; Angular'da resim elemanına **[src]** bağlamasıyla gerçek API görselleri gelecek. Component'lere böl, mantığı ipuçlarıyla sen yaz; bitince göster, birlikte gözden geçirelim.

## ✓ Bu Dersten Çıkarılacaklar

- ✓ **httpResource** gerçek senaryoda iki kez: listede ve id'ye göre detayda.
- ✓ Reaktif URL: detay id'si değişince istek otomatik yenilenir.
- ✓ Sunucudan gelen listeyi **computed** ile yerel süzmek (arama).
- ✓ Her async ekranda **isLoading** / **error** / **hasValue** kalıbı.

## Bölüm

# 🚩 Modül 5 Özeti

Artık Angular'ın reaktiflik motorunu tanıyorsun. Bu modülde durumun nasıl tutulduğunu, türetildiğini ve asenkron verinin nasıl yönetildiğini gördük.

- **Signal:** değişince bağlı her yeri tazeleyen akıllı değer; **set** / **update**.
- **computed:** türetilmiş, salt-okunur, tembel + önbellekli.
- **effect:** yan etkiler için (değer üretmek için değil).
- **linkedSignal:** türetilmiş ama yazılabilir; yerel çalışma kopyası.
- **resource** / **httpResource:** asenkron veriyi signal olarak (value/isLoading/error).
- **viewChild** / **viewChildren:** şablon elemanlarına signal erişimi.

## ❓ Sırada ne var?

**Modül 6 — Directives & Pipes:** şablonların gücünü artıran yapılar. Öğelere davranış ekleyen directive'ler ve veriyi görüntülerken biçimlendiren pipe'lar.

## 🔗 Kendini test et

Şunları cevaplayabiliyor musun? “**computed** ile **linkedSignal** farkı ne? **effect** ne zaman? **resource** 'ta **params** neden önemli?” Cevaplayabiliyorsan mini projeye ve Modül 6'ya hazırsın.

**Takıldığın her yeri sor** — mini projeyi yapıp gösterdiğinde birlikte gözden geçiririz. Kolay gelsin!